Data Science for Public Policy 2026

Lecture 2: R and Python Basics 1

Jun Goto

GRIPS
National Graduate Institute for Policy Studies

23 April 2026

Today's Agenda

1. **Skill check survey**
   What last week's quiz told us about the class's starting point.
2. **Why learn coding in the age of generative AI?**
   When AI helps, when it doesn't, and why coding skill still matters.
3. **R basics, through a real analysis**
   An eight-step walkthrough of a citizen service survey in R.
4. **Homework 1**
   Apply the same workflow to a new dataset on your own.

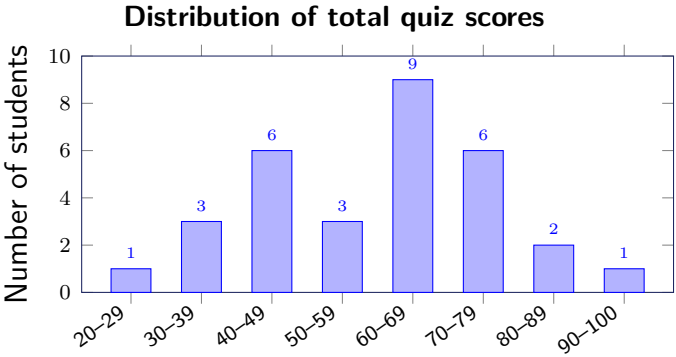
Why I Ran a Skill Check Survey

- The class includes students with **different levels** of coding, statistics, and GitHub experience.
- I want the course to be **challenging but not discouraging**.
- The survey helps identify:
  - prior exposure to R / Python
  - confidence in programming
  - experience with data analysis and GitHub
  - topics students most want to learn
- In other words, the survey is used to **calibrate the course**, not to judge students.

Skill Check Survey: Overall Score Distribution

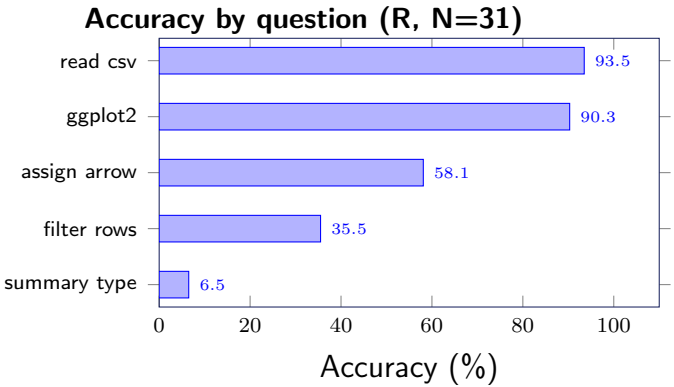
Class snapshot (N=31)

- Mean: **55.2 / 100**
- Median: **60 / 100**
- Highest: **90 / 100**
- Most students in **40–79** range



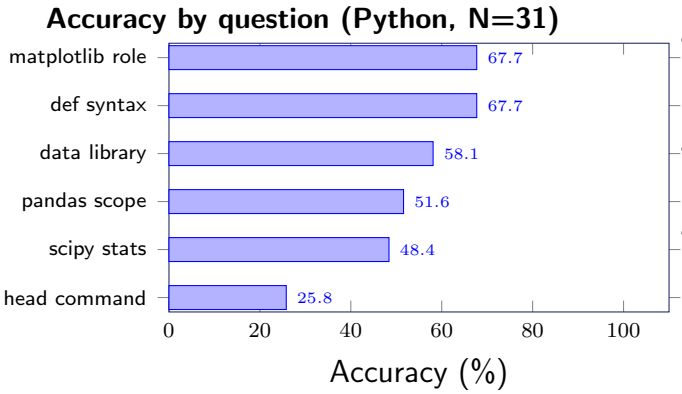
A **mixed but mostly beginner-level** class. We should move carefully through the basics before tackling more advanced ML or NLP tasks.

Skill Check Survey: R Section Results



- `read.csv` / `ggplot2`: package and function **recognition** is strong.
- **Assignment** `<-`: 10 students reversed the arrow (wrote `->`).
- `filter()`: auto-grade said 0%, but 11 students wrote something close; counted generously, ∼35%.
- `summary()` **output type**: the weakest item. Distinguishing *what* the output contains from *what kind of object* it is is still conceptually hard.

Skill Check Survey: Python Section Results



- **Recognition items** (`def` syntax, matplotlib T/F) score around 68%.
- **Role confusion** is the main issue: `matplotlib` is often mistaken for an analysis library (9 students chose it for statistics instead of `scipy`).
- **Writing syntax from scratch** (`df.head()`): only ∼26% could produce the command without being shown options.
- Overall, **Python package roles are less stable** than R package names.

Takeaways and How the Course Will Be Adjusted

What the survey tells us

- R package recognition is relatively strong.
- Python package roles are less stable.
- Writing syntax from scratch is much harder than recognizing it.
- Some free-response items were under-counted by strict auto-grading.

How we will adjust the course

- Start from **syntax**: assignment, functions, data frames.
- **Compare** R and Python concepts side by side.
- Spend time **reading** code, not just writing it.
- Practice **full workflows** end-to-end.
- Treat AI as an **assistant, not a substitute**.

Which Skills This Course Will Especially Develop

Technical skills

- Reading and writing code
- Handling data in R and Python
- Collecting web and text data
- Using ML methods appropriately
- Organizing research with GitHub

Research skills

- Translating policy questions into data tasks
- Checking data quality
- Interpreting output carefully
- Communicating results clearly
- Building replicable research workflows

Why Coding Still Matters in the AI Era

The question is not "Can AI write code for me?"

The better question is: **How can I use AI well?** — and the honest answer still requires coding knowledge.

- AI changes the workflow, but you still need to **define the problem**, **break it into steps**, and **judge the output**.
- Large language models are very good at producing text that **looks** right — but "looks right" is not the same as "is correct".
- AI-generated code can still contain wrong variable names, incorrect assumptions about data structure, subtle statistical mistakes, or fake package functions.
- In policy research, small coding errors can lead to **wrong conclusions**.

Why Coding Still Matters: Break the Problem Down

- A core coding skill is to take a big problem and split it into **small steps**.
- AI works better when the task is **already clear**. A vague prompt gives a weak output.
- A reliable workflow:
  1. define the task
  2. split it into small pieces
  3. write the logic in pseudocode
  4. ask AI to help with the code
- Coding is not only syntax — it is **structured thinking**.

Check the Output: Avoid "Vibe Coding"

Always have a plan to check AI output

Does the code run? Does it fit my data? Does the output make sense? Can I test it on a simple example?

- **"Vibe coding"** = asking AI for large chunks of code without careful checking. It may work for familiar tasks; it is unreliable for new or research-specific ones.
- One interesting finding: people often **feel** more productive with AI even when they are not actually faster.
- Use AI as an assistant, explainer, or draft generator. The **human** still does decomposition, verification, and interpretation.
- If you cannot read and check code, you cannot use AI safely — coding knowledge is what moves you from **blind trust** to **informed use**.

About R

What is R?

- A programming language and environment for statistical computing.
- You type commands into the console; R evaluates them one at a time.
- Save commands in a script to re-run them later.

Why start with R?

- Widely used in social science, public policy, and statistics.
- Strong for data cleaning, summaries, visualization, regression, and reproducible research.
- Today we focus on the **logic of coding**, not on memorizing commands.

Basic idea

Write commands → run commands → inspect results → revise commands

Meet the Dataset: City Service Survey

A city office ran a citizen survey on public service quality. The file `materials/city_service_survey.csv` contains **40 responses**.

id	ward	age	method	wait	sat	recommend
1	North	20s	Online	2	5	Yes
2	North	30s	In-person	5	4	Yes
...	...	...	...	...	...	...
13	East	40s	In-person	10	1	No
18	East	20s	In-person	10	NA	No
...	...	...	...	...	...	...
40	West	50s	Online	6	3	No

Our questions: What is the overall satisfaction? Which ward fares worst? Do longer waits map to lower scores? A few cells are missing — we will handle them carefully.

A Standard Data Analysis Workflow

1. Load the data
2. Inspect the data
3. Compute summary statistics
4. Filter rows by a condition
5. Create a new variable
6. Compare groups
7. Visualize the result
8. Write a substantive interpretation

Each step below introduces the R syntax we need *at that moment*, not in the abstract.

Step 1: Load the Data

```
survey <- read.csv("materials/city_service_survey.csv")
```

Syntax introduced in this line

- `<-` **assigns** the value on the right to a name on the left. (`->` does *not* — 10 students flipped this in the skill check.)
- `read.csv(...)` is a **function call**: arguments go inside the parentheses.
- `"materials/..."` is a **character string** — text must be quoted.
- The result is stored in an **object** called `survey`.

Step 1: What the Console Shows

```
> survey <- read.csv("materials/city_service_survey.csv")
>
```

Assignment runs **silently** — no printout is a *good* sign. Verify by asking R about the object:

```
> dim(survey)
[1] 40 7
> class(survey)
[1] "data.frame"
```

Step 2: Inspect the Data

```r
head(survey) # first 6 rows
str(survey) # structure and data types
summary(survey) # per-column quick summary
nrow(survey); ncol(survey); names(survey)
```

What this teaches

- `survey` is a **data frame**: rows = observations, columns = variables.
- Columns have **types**: `numeric` (`waiting_days`), `character` (`ward`), and may contain `NA` (missing).
- `summary()` returns a named numeric vector for each numeric column — exactly the point of the skill check question.

Step 2: What the Console Shows

```
> head(survey)
  respondent_id ward age_group app_method waiting_days sat_score recommend
1 1 North 20s Online 2 5 Yes
2 2 North 30s In-person 5 4 Yes
3 3 North 40s Online 3 4 Yes
4 4 North 50s Online 4 3 No
5 5 North 30s In-person 6 3 No
6 6 North 60s Online 3 5 Yes

> str(survey)
'data.frame': 40 obs. of 7 variables:
 $ ward : chr "North" "North" "North" ...
 $ age_group : chr "20s" "30s" "40s" "50s" ...
 $ application_method: chr "Online" "In-person" "Online" ...
 $ waiting_days : int 2 5 3 4 6 3 2 5 4 NA ...
 $ satisfaction_score: int 5 4 4 3 3 5 5 4 4 3 ...
 $ ...
```

(Column names shortened for display. `str()` also shows types; look for `NA` in `waiting_days`.)

Step 3: Compute Overall Summary Statistics

```r
survey$satisfaction_score # one column = a vector
mean(survey$satisfaction_score) # returns NA!
mean(survey$satisfaction_score, na.rm = TRUE)
length(survey$satisfaction_score) # counts all rows, including NA
```

Syntax introduced

- `$` extracts one column by name; the column is a **vector**.
- `mean()`, `length()`, `sum()` operate on vectors.
- `NA` **propagates**: the first `mean()` returns `NA` because two values are missing. Use `na.rm = TRUE` to ignore them.

Step 3: What the Console Shows

```
> survey$satisfaction_score
 [1] 5 4 4 3 3 5 5 4 4 3 2 2 1 2 3 1 2 NA 2 3
[21] 5 5 4 4 5 4 5 4 5 5 3 4 2 3 2 NA 2 4 2 3

> mean(survey$satisfaction_score)
[1] NA

> mean(survey$satisfaction_score, na.rm = TRUE)
[1] 3.394737

> length(survey$satisfaction_score)
[1] 40
```

The first `mean()` returns `NA`: R refuses to guess. The `na.rm = TRUE` version returns the correct average over the 38 non-missing values.

Step 4: Filter Rows by a Condition

Which respondents waited a long time?

```
survey$waiting_days >= 7 # a logical vector of TRUE / FALSE
slow_cases <- survey[survey$waiting_days >= 7, ]
nrow(slow_cases)
head(slow_cases)
```

Syntax introduced

- `>=` is a comparison; it returns a **logical vector**, one entry per row.
- `survey[rows, cols]` is the general selection form. Blank means "all".
- A logical vector in the `rows` slot keeps only the `TRUE` rows.

Step 4: What the Console Shows

```
> survey$waiting_days >= 7
 [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE NA
[11] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[21] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[31] FALSE FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE

> nrow(slow_cases)
[1] 15

> head(slow_cases[, c("ward", "waiting_days", "satisfaction_score")])
   ward waiting_days satisfaction_score
NA <NA> NA NA
11 East 9 2
12 East 7 2
13 East 10 1
14 East 8 2
15 East 7 3
```

Why 15 rows, not 14? Row 10's `waiting_days` is `NA`, so the condition returns `NA`, which R fills with a blank `NA` row. Wrapping the condition in `which()` would drop it.

Step 5: Create a New Variable

Add an indicator for a "long wait":

```
survey$long_wait <- survey$waiting_days >= 7
head(survey[, c("ward", "waiting_days", "long_wait")])
```

What this does

- Assigns a **logical** column to `survey$long_wait`.
- You can now use `long_wait` in later analysis just like any other column.
- `c("ward", "waiting_days", "long_wait")` is a character vector used to select multiple columns by name.

Step 5: What the Console Shows

```
> survey$long_wait <- survey$waiting_days >= 7
> head(survey[, c("ward", "waiting_days", "long_wait")])
  ward waiting_days long_wait
1 North 2 FALSE
2 North 5 FALSE
3 North 3 FALSE
4 North 4 FALSE
5 North 6 FALSE
6 North 3 FALSE
```

All six shown rows are `FALSE` because they are from North ward with short waits. Rows from East ward further down the table would show `TRUE`.

Step 6: Compare Groups

```r
options(digits = 3) # show fewer decimals in printouts

aggregate(satisfaction_score ~ ward,
          data = survey, FUN = mean, na.action = na.omit)

aggregate(satisfaction_score ~ long_wait,
          data = survey, FUN = mean, na.action = na.omit)
```

Reading the formula

`y ~ x` means "summarise `y` within groups defined by `x`." `FUN = mean` chooses the summary; `na.action = na.omit` drops missing rows. `options(digits = 3)` trims R's default 6-decimal printout.

Step 6: What the Console Shows

```
> aggregate(satisfaction_score ~ ward,
+ data = survey, FUN = mean, na.action = na.omit)
   ward satisfaction_score
1 East 2.00
2 North 4.00
3 South 4.60
4 West 2.78

> aggregate(satisfaction_score ~ long_wait,
+ data = survey, FUN = mean, na.action = na.omit)
  long_wait satisfaction_score
1 FALSE 4.17
2 TRUE 2.00
```

East is about 2.6 points below **South**. Within-respondent: a long wait is associated with a score about **2.2 points lower** on the 1–5 scale.

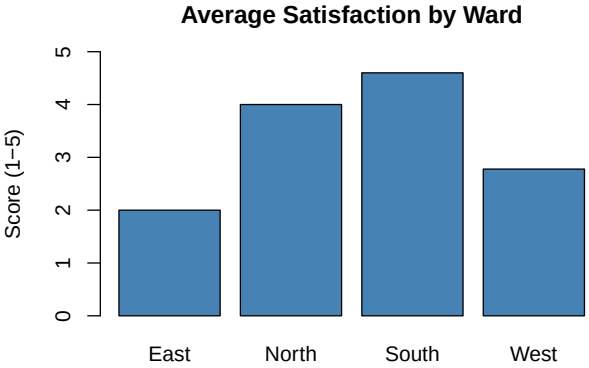
Step 7: Make a Simple Plot

```r
ward_mean <- aggregate(satisfaction_score ~ ward,
                       data = survey,
                       FUN = mean, na.action = na.omit)

barplot(ward_mean$satisfaction_score,
        names.arg = ward_mean$ward,
        ylim = c(0, 5),
        col = "steelblue",
        main = "Average Satisfaction by Ward",
        ylab = "Score (1-5)")
```

Build a small summary table first, *then* plot it. Arguments such as `names.arg`, `ylim`, `main`, and `ylab` control what the reader sees.

Step 7: The Plot



East ward has by far the lowest average; South has the highest. The pattern matches what we saw numerically in Step 6.

Step 8: Interpret the Result

What we find in this dataset

- Overall average satisfaction is about **3.4 / 5**.
- **East** has the lowest average (≈ 2.0) and the longest waits; **South** is the opposite (≈ 4.6).
- Respondents with `long_wait = TRUE` score on average about **2.0**, versus about **4.2** for shorter waits.

A good interpretation moves from **commands** to a **substantive statement**:

> *"Service delays appear to be a key driver of dissatisfaction, and East ward is the most affected. A targeted review of East ward's processing times would be a sensible next step."*

Common Beginner Mistakes in R

Syntax and typing

- Misspelling object or column names
- Forgetting quotation marks around text
- Using = and <- carelessly
- Mismatched parentheses or quotes

Data and output

- Not checking whether a column is numeric or character
- Ignoring `NA` values in `mean()` / `sum()`
- Running code without reading the output
- Assuming a plot is right without reading the numbers behind it

Errors are normal. The goal is to learn how to **read** them and fix them.

A Good Habit: Check One Step at a Time

Do not write twenty lines and then hope everything works. Instead, cycle through four actions for *each* small block of code:

1. Run one short block	→	**2. Inspect** the output	→	**3. Confirm** it makes sense	→	**4. Move on** to the next block

This habit is useful whether you code alone or with AI assistance — and it is exactly what we practiced in today's input/output slide pairs.

Homework 1: R Basics Practice

Assignment

Reproduce today's 8-step workflow on a **new dataset**: student reading scores across four countries (Korea, Germany, USA, Brazil). The questions are different, so you will need to adapt the lecture code, not copy it.

- Dataset + full instructions: `lectures/02_r_and_python_basics_1/assignment/`
- Submit **one R script** + a **short written interpretation** (~half a page, Markdown or PDF)
- **Due: Wednesday, 29 April 2026, 12:00 PM**

Before you start

Make sure R and RStudio run on your computer. Run one short block at a time and check the output. If a line does not run, bring the exact error message to class.

Key Takeaways

About the course

- Calibrated to the class's actual skill profile.
- Generative AI is useful — but coding knowledge is what lets you use it well.
- Goal is not memorisation; it is **building confidence**.

About R

- Store objects with `<-`.
- Inspect data before analysing it.
- Summarise, then visualise, then interpret.
- Handle `NA` deliberately, not by accident.

Next step: Homework 1 — apply this exact workflow to a new dataset.